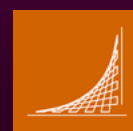




DOSW COMPANY



ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 7

GRUPO 1

ANALISIS CORTE #1





!!!!FELICITACIONES POR TERMINAR SU
PROCESO DE ONBOARDING !!!!



ANALISIS CORTE 1

Rendimiento del Curso

Uso de la
IA
Parcial Corte
#1
Progreso
Inicial



PLAN DE MEJORA - NUEVAS REGLAS DEL CURSO



COMO VAN A SER EVALUADOS?

CORTE 2

- 40% Parcial (teorico + practico)
- 30% Proyecto Engineering Assignment
- 10% Quices y Actividades Semanales
- 10% Bitacora – Sesiones
- 10% Refuerzo – Talent Readiness

QUICES TODAS LAS SEMANAS

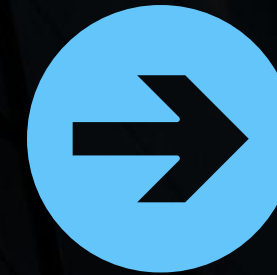
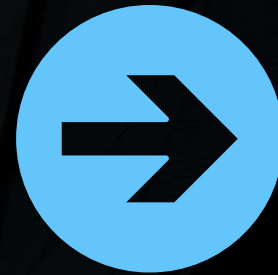
TALENT READINESS

Grupos de Refuerzo de temas del corte, ejercicios desde el primer tema hasta el tema de semana 6, estan disponibles en Teams - MATERIAL DE REFUERZO

EQUIPO CHIMCHAR



EQUIPO PIPLUP



TALENT READINESS



EQUIPO
CHIMCHAR

+0,3

EQUIPO
PIPLUP

+0,2



+0,5



EQUIPO
CHIMCHAR

-0,1

EQUIPO
PIPLUP

-0,05



-0,5





RECUERDEN

Les va a llegar un correo, sobre en cual grupo de refuerzo estan, el refuerzo inicia desde HOY



CORTE #2

ENGINEERING
ASSIGNMENT



DOSW COMPANY

SET DE PRUEBAS

TRANSFORMAR LOS
CRITERIOS DE
APROBACION

TIPOS DE COBERTURA

1 Cobertura de Líneas

- Mide qué porcentaje de líneas de código fueron ejecutadas.

Ejemplo:

Si solo pruebas el caso **edad = 20**, la línea del **else** nunca se ejecuta.

```
if (edad >= 18) {  
    return "Mayor de edad";  
} else {  
    return "Menor de edad";  
}
```



Cobertura ≠ 100%

2 Cobertura de Ramas

- Mide si se **ejecutaron** todas las decisiones posibles (**if/else**, **switch**, ternarios).
- Es más **exigente** que la cobertura de líneas.



3 Cobertura de Métodos

- Mide si los **métodos** fueron **invocados** al menos una vez.



4 Cobertura de Clases

- Mide si las **clases** fueron **utilizadas** durante las pruebas.





EJEMPLO

```
function calcularDescuento(precio, esClienteVIP) {  
  if (precio > 100) {  
    if (esClienteVIP) {  
      return precio * 0.8;  
    } else {  
      return precio * 0.9;  
    }  
  } else {  
    return precio;  
  }  
}
```

```
function contarHasta(n) {  
  let total = 0;  
  for (let i = 0; i < n; i++) {  
    total += i;  
  }  
  return total;  
}
```

```
function esPar(numero) {  
  return numero % 2 === 0;  
}
```

TIPOS DE COBERTURA

Cobertura de ramas (Branch Coverage)

Cada rama posible de una decisión (*if / else*)

Ej: ¿Cuáles serían las ramas de la función *calcularDescuento*?

```
function calcularDescuento(precio, esClienteVIP) {  
  if (precio > 100) {  
    if (esClienteVIP) {  
      return precio * 0.8;  
    } else {  
      return precio * 0.9;  
    }  
  } else {  
    return precio;  
  }  
}
```

```
calcularDescuento(150, true);    // precio > 100, VIP  
calcularDescuento(150, false);  // precio > 100, no VIP  
calcularDescuento(80, false);   // precio <= 100
```

Cobertura de funciones (Function Coverage)

Mide si cada función ha sido llamada al menos una vez.

Ej: ¿Cuáles serían las funciones?

```
calcularDescuento(200, false);  
esPar(3);  
contarHasta(1);
```

TIPOS DE COBERTURA

Cobertura de condiciones (*Condition Coverage*)

Mide cada condición booleana dentro de una decisión evalúa a true y false.

```
calcularDescuento(120, true); // true, true  
calcularDescuento(120, false); // true, false  
calcularDescuento(90, true); // false, true
```

Caminos en `calcularDescuento`:

1. `precio > 100` → `esClienteVIP` → descuento 20%
2. `precio > 100` → `!esClienteVIP` → descuento 10%
3. `precio <= 100`

Cobertura de caminos (*Path Coverage*)

Mide si se ejecutan todos los caminos posibles del programa.

Cobertura de líneas (*Line Coverage*)

Qué porcentaje de líneas de código se ejecutan al menos una vez.

```
calcularDescuento(200, false);  
esPar(3);  
contarHasta(1);
```



TIPOS DE COBERTURA

Cobertura de ciclos (*Loop Coverage*)

Mide si los ciclos se ejecutan 0 o más veces

```
contarHasta(0); // 0 iteraciones  
contarHasta(1); // 1 iteración  
contarHasta(5); // múltiples iteraciones
```

TIPOS DE COBERTURA - EJERCICIO

</> La función bajo análisis

■ líneas ejecutables (cobertura de líneas) ■ puntos de decisión (cobertura de ramas)

```
1 public class PlatoService {  
2     public String calcularDescuento(Plato plato, int cantidad) {  
3         if (plato == null) {  
4             throw new IllegalArgumentException("Plato no puede ser nulo");  
5         }  
6         if (cantidad >= 10) {  
7             return "20% descuento por volumen";  
8         }  
9         if (plato.getPrecio() > 50000) {  
10            return "10% descuento premium";  
11        }  
12        return "Sin descuento";  
13    }  
14 }
```

TIPOS DE COBERTURA - EJERCICIO

PREGUNTA A

¿Cuántas líneas ejecutables tiene el método?

No cuentan llaves solas ni la declaración de la clase.

3 líneas

5 líneas

6 líneas

8 líneas

PREGUNTA A

¿Cuántas líneas ejecutables tiene el método?

No cuentan llaves solas ni la declaración de la clase.

3 líneas

5 líneas

6 líneas

8 líneas

Correcto. Líneas 2, 3, 4, 7, 10 y 12 son ejecutables.

PREGUNTA B

¿Cuántos puntos de decisión (ramas) tiene el método?

Cuenta cada if independiente.

1 rama

2 ramas

3 ramas

4 ramas

PREGUNTA B

¿Cuántos puntos de decisión (ramas) tiene el método?

Cuenta cada if independiente.

1 rama

2 ramas

3 ramas

4 ramas

Correcto. Tres if independientes: null check (L3), cantidad >= 10 (L6), precio > 50000 (L9). Cada uno tiene rama true y rama false.

TIPOS DE COBERTURA - EJERCICIO

PREGUNTA C

Si solo pruebas con cantidad = 15 y precio = 30000, ¿qué cobertura de líneas obtienes?

El método no lanza excepción, retorna en línea 7.

100%

~83%

~50%

~67%

PREGUNTA C

Si solo pruebas con cantidad = 15 y precio = 30000, ¿qué cobertura de líneas obtienes?

El método no lanza excepción, retorna en línea 7.

100%

~83%

~50%

~67%

Correcto. Ejecutas líneas 2, 3, 6, 7 (4 de 6 ejecutables).
L4 (throw) y L10, L12 nunca se alcanzan = ~67%.

PREGUNTA D

Con ese mismo test, ¿puedes afirmar que tienes 100% de cobertura de ramas?

Piensa en todos los caminos posibles de cada if.

Sí, la tengo

No, me faltan ramas

PREGUNTA D

Con ese mismo test, ¿puedes afirmar que tienes 100% de cobertura de ramas?

Piensa en todos los caminos posibles de cada if.

Sí, la tengo

No, me faltan ramas

Correcto. Solo cubres la rama true del if de cantidad >= 10. Las ramas false de L6, L9, y el if de null siguen sin probar.

TIPOS DE COBERTURA - EJERCICIO

Arrastra o selecciona el tipo de cobertura que cubre cada test. Haz clic en el badge para asignarlo.

TEST	VALORES	RESULTADO ESPERADO	¿QUÉ COBERTURA APORTA?
T1	plato=null	lanza IllegalArgumentException	líneas ramas método clase
T2	cantidad=15, precio=30000	"20% descuento por volumen"	líneas ramas método clase
T3	cantidad=2, precio=60000	"10% descuento premium"	líneas ramas método clase
T4	cantidad=1, precio=20000	"Sin descuento"	líneas ramas método clase

TIPOS DE COBERTURA - EJERCICIO

TEST	VALORES	RESULTADO ESPERADO	¿QUÉ COBERTURA APORTA?
T1	plato=null	lanza IllegalArgumentException	líneas ramas método clase Perfecto. T1 ejecuta L2, L3, L4 (líneas), cubre la rama true del null-check (ramas), invoca el método por primera vez (método), e instancia la clase (clase).
T2	cantidad=15, precio=30000	"20% descuento por volumen"	líneas ramas método clase Correcto. T2 ejecuta nuevas líneas (L6, L7) y cubre la rama true de cantidad>=10. Método y clase ya estaban cubiertos por T1.
T3	cantidad=2, precio=60000	"10% descuento premium"	líneas ramas método clase Correcto. T3 llega a L9 (nueva línea) y cubre la rama true de precio>50000. Es el único test que ejecuta L10.
T4	cantidad=1, precio=20000	"Sin descuento"	líneas ramas método clase Correcto. T4 es el único que llega a L12 (return "Sin descuento"), completando la cobertura de líneas y las ramas false de L6 y L9.



DOSW COMPANY

SISTEMAS



SISTEMAS CON ESTADO

Un sistema con estado guarda la informacion del cliente entre las solicitudes

Es decir el servidor recuerda el contexto de las interacciones anteriores y lo usa en los siguientes

SISTEMAS SIN ESTADO

Un sistema sin estado no guarda informacion del cliente entre solicitudes

Cada peticion es independiente y debe contener toda la informacion necesaria para ser procesada



SISTEMAS CON ESTADO O SIN ESTADO?

Una aplicacion web que usa sesiones de login

CON ESTADO

Juego en linea MMORPG

CON ESTADO

Carritos de compra

CON ESTADO

Un restaurante de comida rapida

SIN ESTADO



DOSW COMPANY

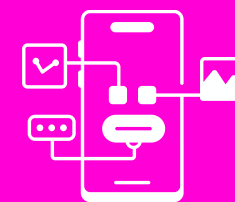
ARQUITECTURA DE SERVICIOS

EL CLIENTE PIDE - EL SERVIDOR RESPONDE



Es un modelo de diseño de software donde las tareas se dividen entre dos partes: el cliente que solicita un servicio o recurso, y el servidor que procesa dicha petición y entrega la respuesta

CLIENTE



→ REQUEST

← RESPONSE

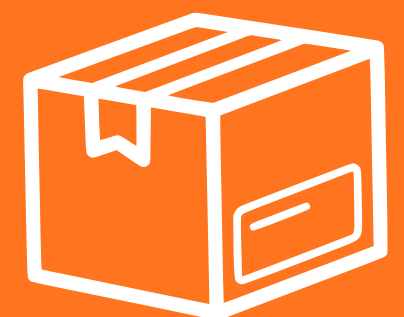
SERVIDOR

LOGICA DE
NEGOCIO

BASE DE DATOS

AUTENTICACION

RECURSOS



3 REGLAS DEL PATRON C-S



El cliente PIDE, el servidor RESPONSE,
Nunca al reves



El servidor no recuerda al cliente
(stateless)

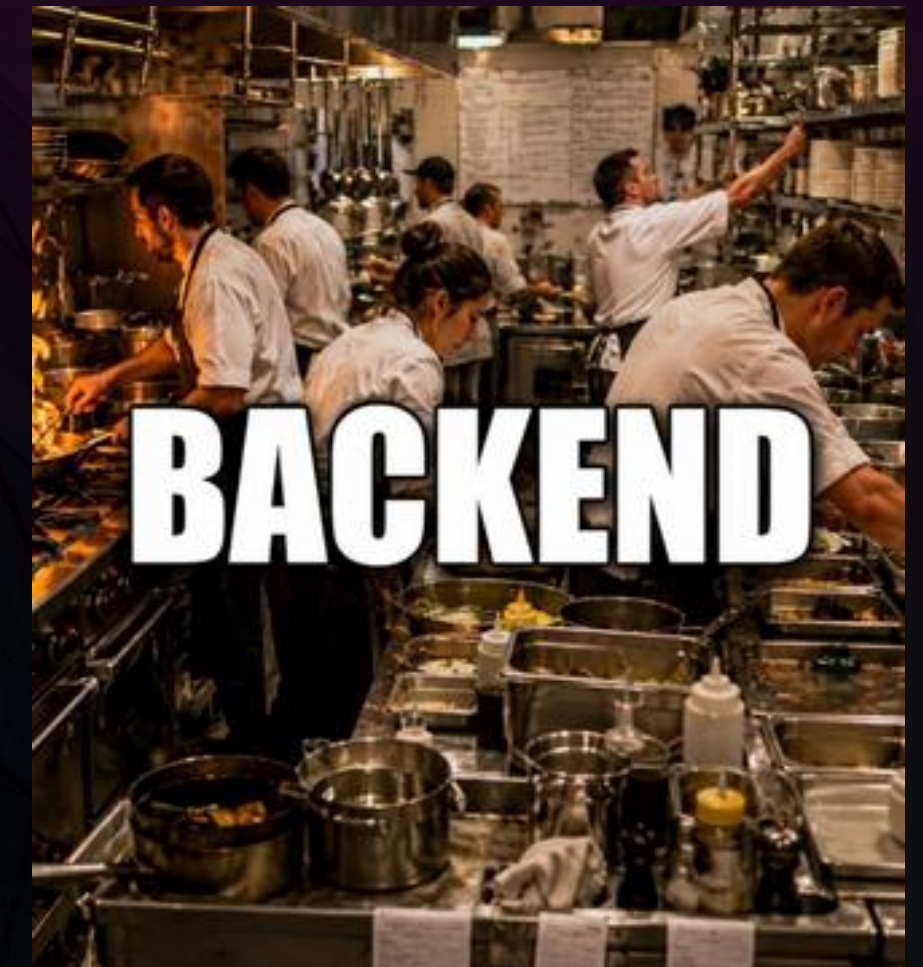


Muchos clientes hablan con el mismo
servidor

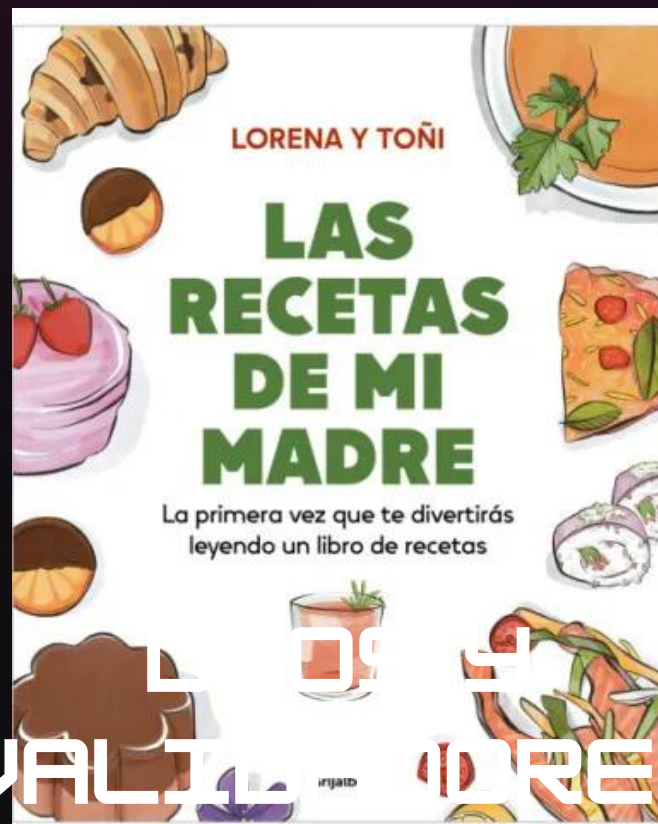
ANALOGIA DEL MUNDO REAL



ANALOGIA DEL MUNDO REAL



ANALOGIA DEL MUNDO REAL



ANALOGIA DEL MUNDO REAL



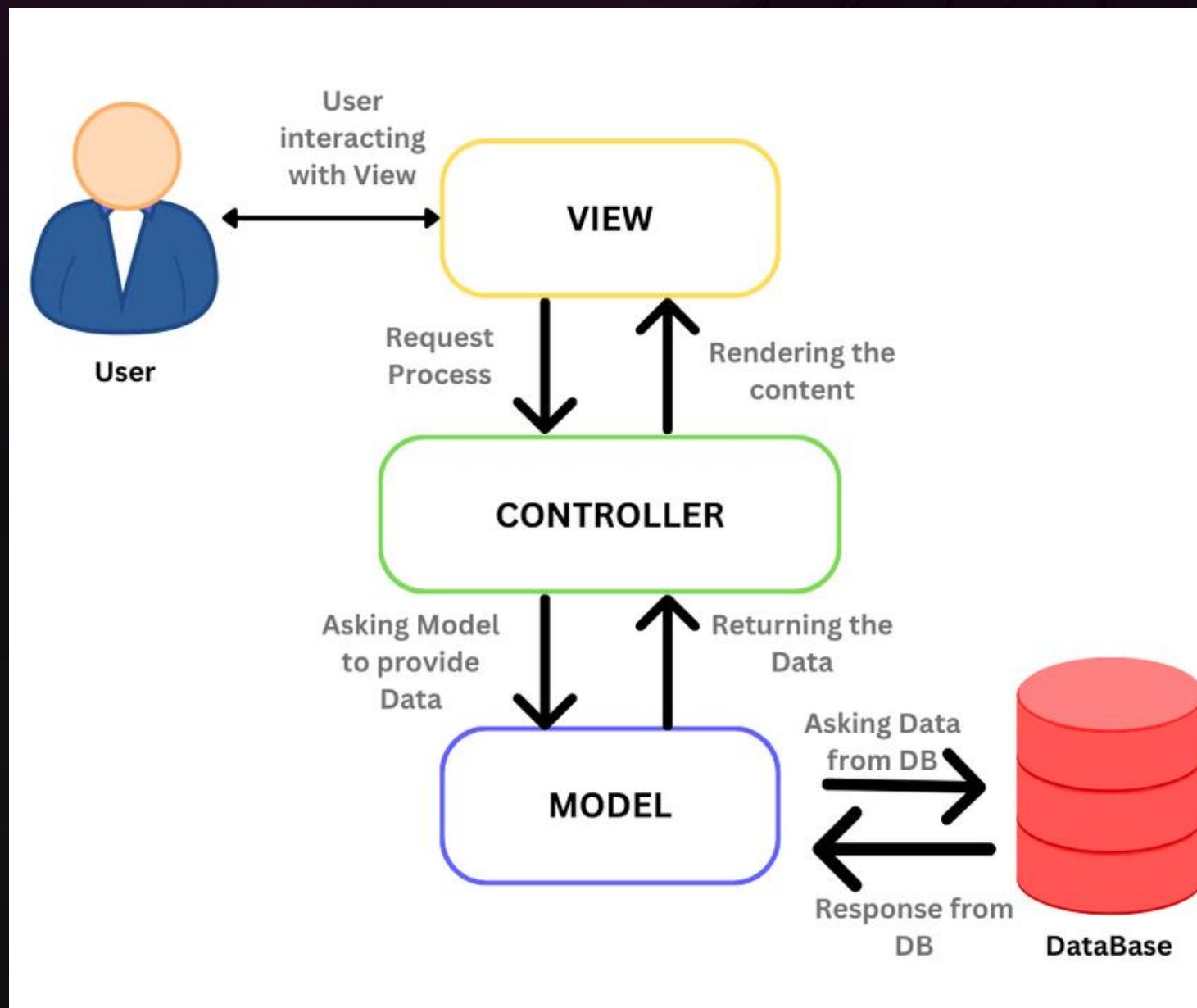
QUE VAMOS A VER ESTE CORTE





COMO SE ORGANIZA LA COCINA POR DENTRO?

MODELO VISTA CONTROLADOR (MVC)



Es un patron de software que divide una aplicacion en tres componentes interconectados

MODELO: Representa la informacion (datos) y las reglas de negocio de la aplicacion - Gestiona, almacena y manipula los datos

VISTA: Capa de la presentacion de los datos al usuario - Define como se muestra la informacion en pantalla

CONTROLADOR: Es el intermediario que gestiona la interaccion entre el usuario, el modelo y la vista - Invoca la logica de negocio



TRES CAPAS - TRES RESPONSABILIDADES

CONTROLLER

El mesero

Recibe la petición Con HTTP
Extrae los datos del request
Llama al Service
Devuelve el Response Dto
No tiene lógica de Negocio

SERVICE

La Cocina

Contiene lógica de negocio
Valida las reglas del dominio
Llama al Repository
Transforma datos (Entidad $\leftrightarrow$ DTO)
NO conoce HTTP

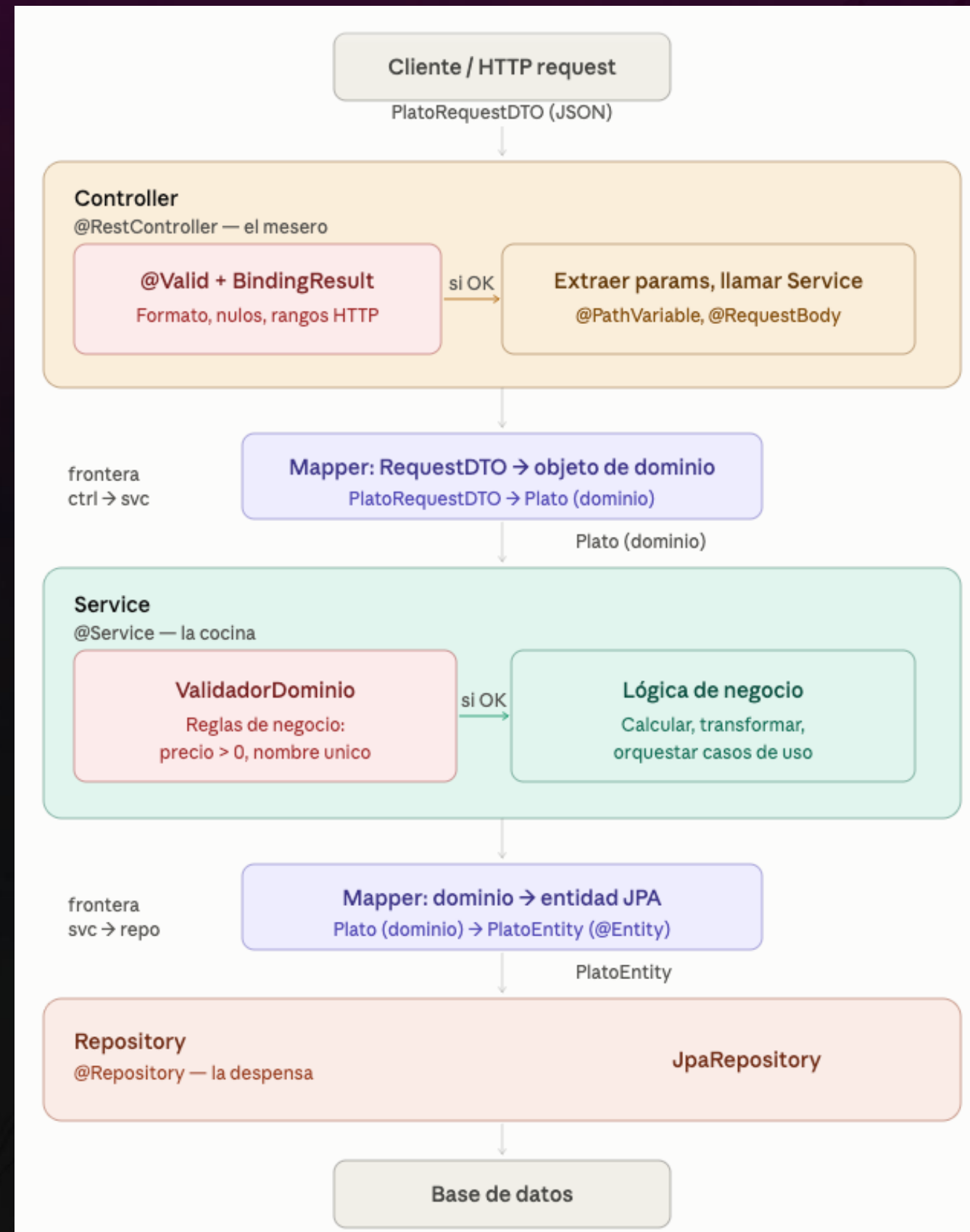
REPOSITORY

La Despensa

Accede a la base de datos
Guarda y recupera datos
Solo habla SQL/JPA
No tiene lógica propia
No conoce la API

CLIENT

El proveedor Externo
Accede a servicios Externos
Lógica de Abstracciones



ARQUITECTURA EN CAPAS - ESTRUCTURA INICIAL

COMO LO VERIAMOS EN NUESTROS PROYECTOS?

```
SRC/MAIN/JAVA
└─ com.restaurante
    └─ RestauranteApplication.java @SpringBoot
    └─ controller/
        ├── PlatoController.java Controller
        └─ GlobalExceptionHandler.java Controller
    └─ service/
        ├── PlatoService.java Service
        └─ PlatoServiceImpl.java Service
    └─ repository/
        └─ PlatoRepository.java Repository
    └─ model/
        ├── domain/
        │   └─ Plato.java Dominio
        ├── entity/
        └─ dto/
            ├── request/
            │   └─ PlatoRequestDTO.java Controller
            └─ response/
                └─ PlatoResponseDTO.java Controller
    └─ mapper/
        ├── PlatoMapper.java Dominio
        └─ PlatoEntityMapper.java Dominio
    └─ validator/
        └─ PlatoValidator.java Service
    └─ exception/
        ├── PlatoNotFoundException.java Dominio
        └─ PlatoAlreadyExistsException.java Dominio
    └─ config/
        ├── SwaggerConfig.java Config
        └─ CorsConfig.java Config
```

```
SRC/MAIN/JAVA
└─ com.restaurante
    ├── controller/
    ├── service/
    ├── repository/
    ├── mapper/
    ├── validator/
    └─ exception/

SRC/TEST/JAVA ← ESPEJO EXACTO
└─ com.restaurante
    ├── controller/
    │   └─ PlatoControllerTest.java @WebMvc
    ├── service/
    │   └─ PlatoServiceImplTest.java Unit
    ├── repository/
    │   └─ PlatoRepositoryTest.java @WebMvc
    ├── mapper/
    │   └─ PlatoMapperTest.java Unit
    ├── validator/
    │   └─ PlatoValidatorTest.java Unit
    ├── exception/
    │   └─ GlobalExceptionHandlerTest.java @WebMvc
    └─ integration/

SRC/TEST/RESOURCES
└─ application-test.properties Config
```



VAMOS A CONSTRUIR
NUESTRO
RESTAURANTE DEL
LABORATORIO 3



DOSW COMPANY

ESTRUCTURAS DE SOFTWARE AL DETALLE



CONTROLLER

Es la unidad de software que recibe las solicitudes HTTP (request) y devuelve las respuestas (response).
Se comunica con los services, nunca directamente con la base de datos.

¿COMO SE GENERA?

Se anota con @RestController y se definen sus endpoints con @RequestMapping o @GetMapping o @PostMapping

```
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.save(user);
    }
}
```



COMPONENT

Es una unidad de software generica que establece que es una clase de utilidad para el aplicativo

¿COMO SE GENERA?

Se anota con @Component

```
import org.springframework.stereotype.Component;

@Component
public class EmailValidator {

    public boolean isValid(String email) {
        return email != null && email.contains("@");
    }
}
```



SERVICE

Unidad de software que se encarga de tener reglas complejas y logica de negocio en la capa de servicios

¿COMO SE GENERA?

Se anota con @Service, se inyecta en el controlador y es el encargado de llamar al repository para el acceso a BD

```
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<User> findAll() {
        return userRepository.findAll();
    }

    public User save(User user) {
        // Ejemplo de lógica de negocio
        if(user.getEmail() == null || user.getEmail().isEmpty()) {
            throw new IllegalArgumentException("El email no puede estar vacío");
        }
        return userRepository.save(user);
    }
}
```



REPOSITORY

Unidad de software que se encarga de gestionar las operaciones relacionadas a la capa de persistencia

¿COMO SE GENERA?

Es una interface que extiende una Implementacion que nos otorga metodos CRUD sin necesidad de implementarlos manualmente, cuando la consulta requiere de algo mas especializado se puede agregar dentro de la implementacion de esta interface

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    // Aquí puedes agregar consultas personalizadas si lo necesitas
    User findByEmail(String email);
}
```

Se puede usar en relacional y no relacional



DTOS - RECORDS Y ENUMERACIONES



DTOS

Son objetos simples que sirven para transportar datos entre capas, no contienen logica de negocio, evitan exponer directamente las entidades de base de datos y permiten controlar que informacion se envia y se recibe de la API

```
// DTO para devolver datos de usuario al cliente
public class UserDto {
    private Long id;
    private String name;
    private String email;

    // Constructor, getters y setters
    public UserDto(Long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    // Getters y setters...
}
```



RECORDS

Son una forma mas concisa de crear DTOS o clases inmutables, ya que automaticamente generan: constructor, getters, equals, hashCode y toString

Son ideales para DTOS y respuestas JSON en APIS

```
public record UserRecord(Long id, String name, String email) {}
```

```
@GetMapping("/{id}")  
public UserRecord getUser(@PathVariable Long id) {  
    User user = userService.findById(id);  
    return new UserRecord(user.getId(), user.getName(), user.getEmail());  
}
```



ENUMERACIONES

Son tipos especiales para representar un conjunto fijo de constantes.
Se usan mucho en APIs para definir estados, roles o categorías.

```
public enum OrderStatus {  
    PENDING,  
    PAID,  
    SHIPPED,  
    DELIVERED,  
    CANCELLED  
}
```

```
@Entity  
public class Order {  
    @Id  
    private Long id;  
  
    private String customer;  
  
    @Enumerated(EnumType.STRING) // Guarda "PENDING", "PAID", etc. en la BD  
    private OrderStatus status;  
}
```

```
public record OrderDto(Long id, String customer, OrderStatus status) {}
```



DOSW COMPANY

ENTIDADES DE NEGOCIO VS MODELOS



ENTIDADES DE NEGOCIO

Representan los objetos centrales del dominio, son la fuente o insumo en la logica de negocio, tienen reglas de negocio y comportamiento, no dependen del almacenamiento de los datos

```
public class Order {  
    private Long id;  
    private String customerId;  
    private OrderStatus status;  
    private List<OrderItem> items;  
  
    public void markAsPaid() {  
        if (this.status != OrderStatus.PENDING) {  
            throw new IllegalStateException("La orden no puede marcarse como pagada");  
        }  
        this.status = OrderStatus.PAID;  
    }  
  
    public double calculateTotal() {  
        return items.stream()  
            .mapToDouble(OrderItem::getSubtotal)  
            .sum();  
    }  
}
```



ENTIDADES

Representan como se guardan los datos en la BD, si es relacional usan la anotacion @Entity, si no es relacional utilizan la anotacion @Document, No tiene logica de negocio ya que su fin es reflejar la estructura de la BD

```
@Entity
@Table(name = "orders")
public class OrderEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String customerId;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    // getters y setters
}
```



VALIDADORES VS UTILIDADES VS MAPPERS



VALIDADORES

Un validador es una clase o componente cuyo objetivo es verificar que los datos cumplen con ciertas reglas de negocio o de formato antes de ser procesados.

Se encargan de verificar entradas (Request DTOs, entidades, etc.).
Generalmente se usan con anotaciones (@NotNull, @Email, @Size, etc.) o con lógica personalizada.

```
@Component
public class AgeValidator implements ConstraintValidator<ValidAge, Integer> {
    @Override
    public boolean isValid(Integer age, ConstraintValidatorContext context) {
        return age != null && age >= 18 && age <= 100;
    }
}
```

```
public class UserRequest {
    @NotNull(message = "El nombre no puede ser nulo")
    @Size(min = 3, message = "El nombre debe tener al menos 3 caracteres")
    private String name;

    @Email(message = "El correo no es válido")
    private String email;

    @Min(value = 18, message = "Debe ser mayor de edad")
    private int age;

    // getters y setters
}
```



UTILIDADES

Las utilidades son clases estáticas que contienen funciones genéricas y reutilizables que no dependen directamente de reglas de negocio ni de un estado de la aplicación.

No manejan persistencia, ni lógica de negocio compleja.

Se utilizan para operaciones comunes como: formatear fechas, generar UUIDs, cifrar, etc.

Son estáticas, no necesitan inyección de dependencias.

```
public class DateUtils {  
    private DateUtils() {} // constructor privado para evitar instancias  
  
    public static String formatDate(LocalDate date) {  
        return date.format(DateTimeFormatter.ofPattern("dd/MM/yyyy"));  
    }  
  
    public static LocalDate parseDate(String date) {  
        return LocalDate.parse(date, DateTimeFormatter.ofPattern("dd/MM/yyyy"));  
    }  
}
```



MAPPERS

Los mappers se encargan de transformar datos entre diferentes capas o modelos.

Usados para convertir entre DTOs y Entidades.

Se pueden hacer manualmente o usando librerías como MapStruct o ModelMapper.

Mantienen limpio el código, evitando mezclar lógica de conversión dentro de los servicios o controladores.

```
@Component
public class UserMapper {

    public User toEntity(UserRequest dto) {
        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        user.setAge(dto.getAge());
        return user;
    }

    public UserResponse toResponse(User entity) {
        return new UserResponse(entity.getId(), entity.getName(), entity.getEmail());
    }
}
```

```
@Mapper(componentModel = "spring")
public interface UserMapper {
    User toEntity(UserRequest dto);
    UserResponse toResponse(User entity);
}
```



MODULO DE DIAGRAMACION # 3



DIAGRAMA DE COMPONENTES GENERAL

- Muestra la arquitectura global del sistema, como se conecta con otros sistemas externos que interactúan entre sí
- Define responsabilidades y relaciones
- Agrega dependencias Externas
- Agrupa por dominio si es necesario

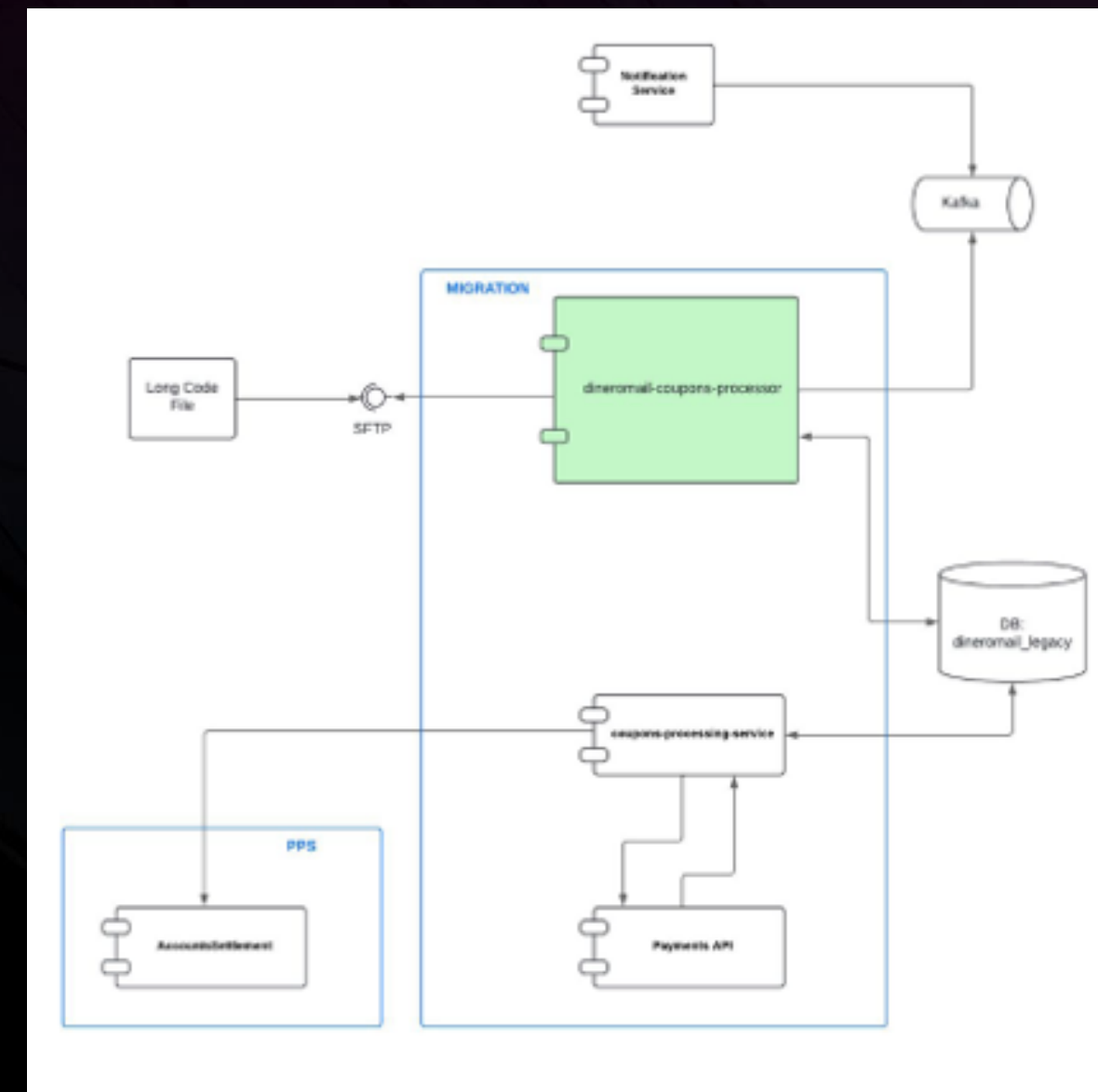
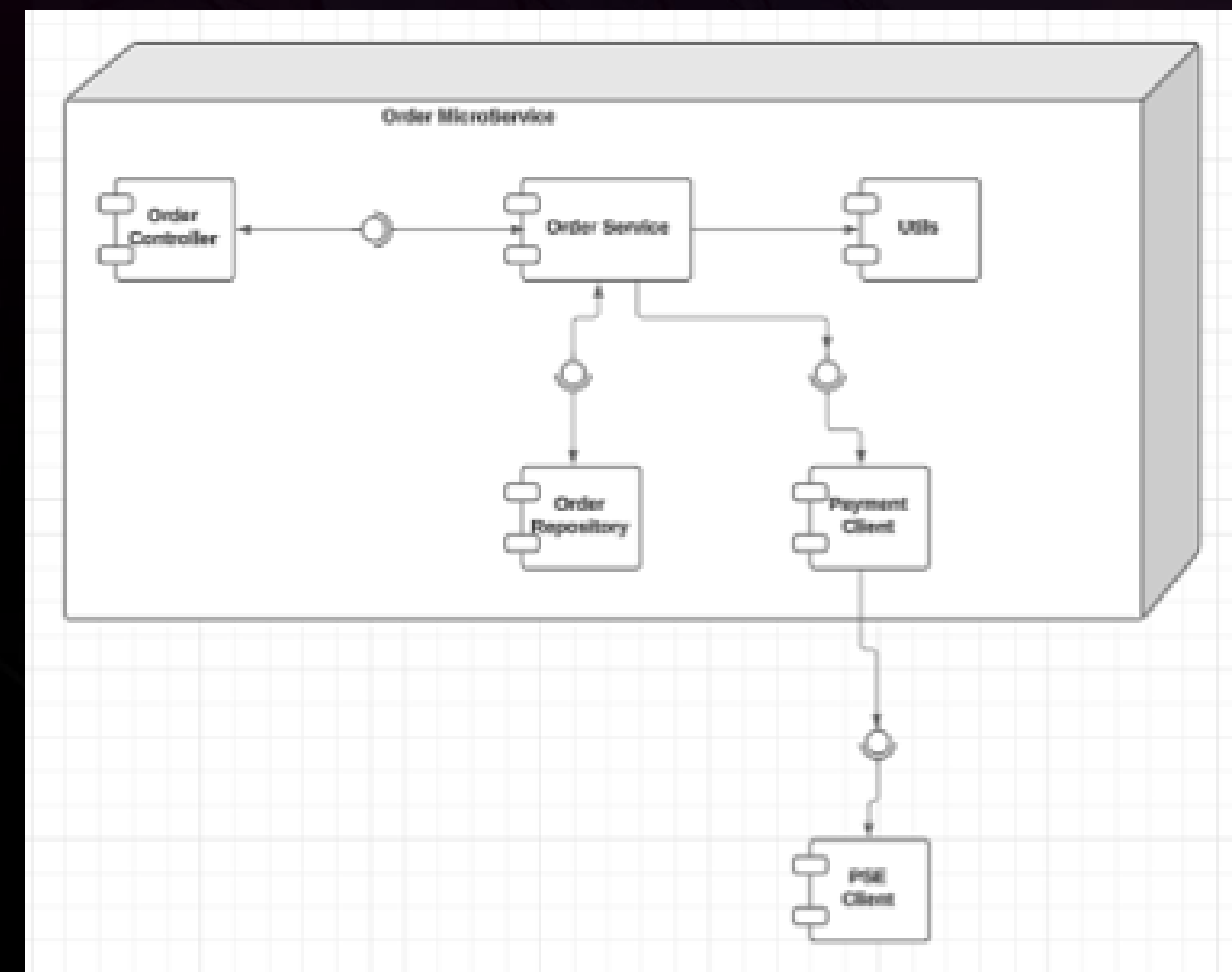
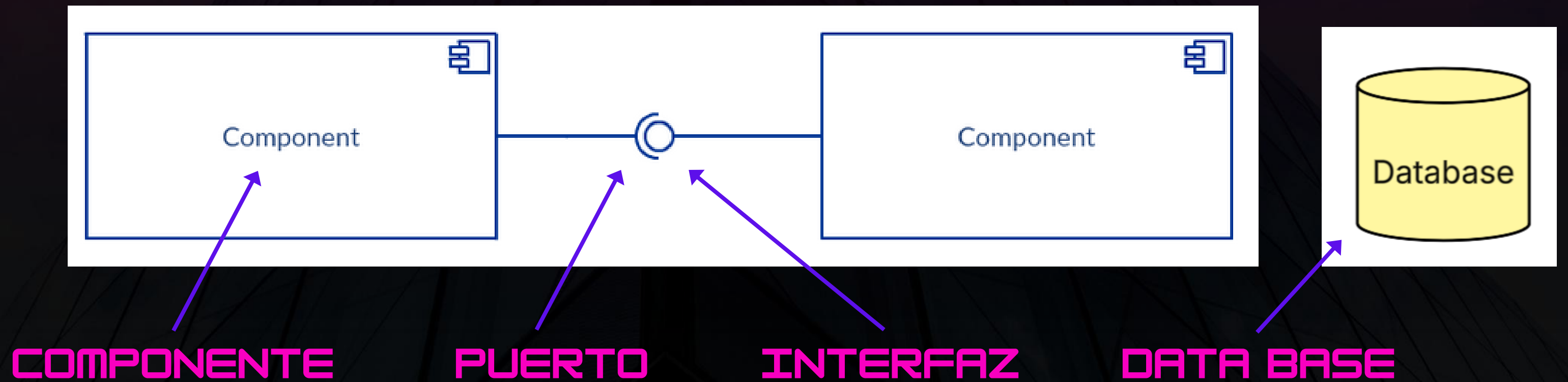


DIAGRAMA DE COMPONENTES ESPECIFICO

- Muestra la arquitectura dentro de un aplicativo, es decir, cómo están organizados sus módulos principales.
- Define el propósito del servicio específico
- Identifica los componentes internos:
- Controladores, Servicios o lógica de negocio, repositorio o acceso a datos, modelos/Entidades, Base de datos o cliente externo, adaptadores o integraciones
- Dibuja los módulos y relaciones



SIMBOLOS DEL DIAGRAMA





COMO NOS QUEDARIAN
NUESTROS DIAGRAMAS DE
COMPONENTES GENERAL Y
ESPECIFICO DEL
RESTAURANTE ?



DISEÑO DE APIS WEB



API Y API REST

API (Application Programming Interface) representando un conjunto de procedimientos que sirven para comunicar 2 o mas aplicaciones entre ellas

Intermediario a travez del cual podemos utilizar otros productos o servicios sin saber nada de su implementacion

REST (Representational State Transfer), es una forma sencilla de organizar interacciones entre sistemas independientes e indica un conjunto de principios arquitectonicos

- Arquitectura Cliente-Servidor
- Comunicacion Stateless
 - Caching
 - Sistema a Niveles
- Interfaz uniforme para los componentes
- Codigo on Demand

**NO TODAS LAS API SON REST, PERO
TODOS LOS SERVICIOS REST SON API**

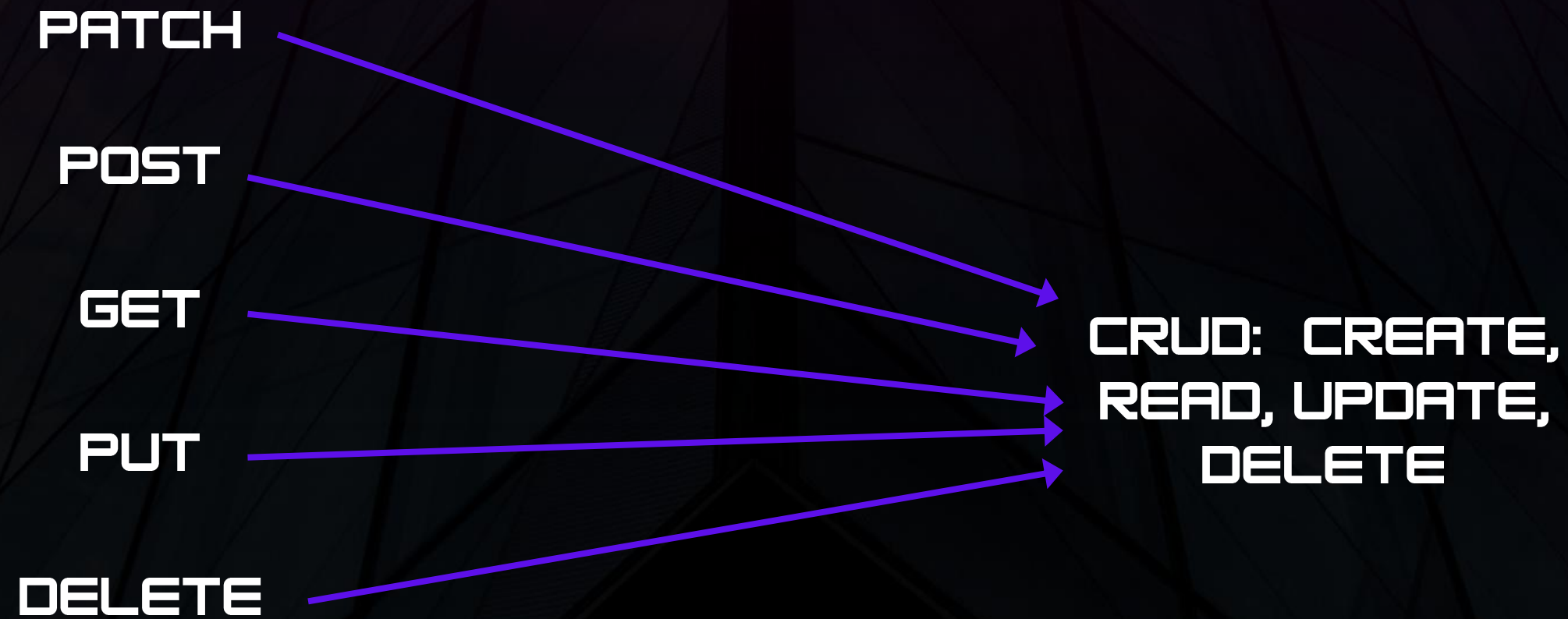
PERO NO ES LA UNICA FORMA





VERBOS HTTP

Pero más allá de estos principios, el elemento fundamental del enfoque REST son los recursos. Por recursos se entiende cualquier dato o información recuperable de manera única. En el ámbito Web el mecanismo utilizado para recuperar los recursos es el uso de URI que se basan en llamadas HTTP.



EL HTTP SIN EMBARGO ES EL PROTOCOLO QUE PERMITE EL ENVÍO CONCRETO DE ESTOS DATOS DE UN LADO A OTRO EN LA WEB.



VERBOS HTTP

Se utilizan para comunicar al servidor que hacer con ese recurso en especifico

POST

Se utiliza para comunicar al servidor que se debe causar el procesamiento del cuerpo de la solicitud, generando nuevos recursos

GET

Permite Indicar al servidor que transmita los datos identificados al cliente, es solo de lectura y obtencion de la informacion

PUT

Se realiza cuando se desea crear o actualizar en totalidad el recurso identificado en la URL

DELETE

Debe utilizarse cuando se desee eliminar el recurso identificado por la URL de la solicitud

PATCH

Se usa para aplicar modificaciones parciales a un recurso existente, es decir solo algunas de sus propiedades



VERBOS HTTP

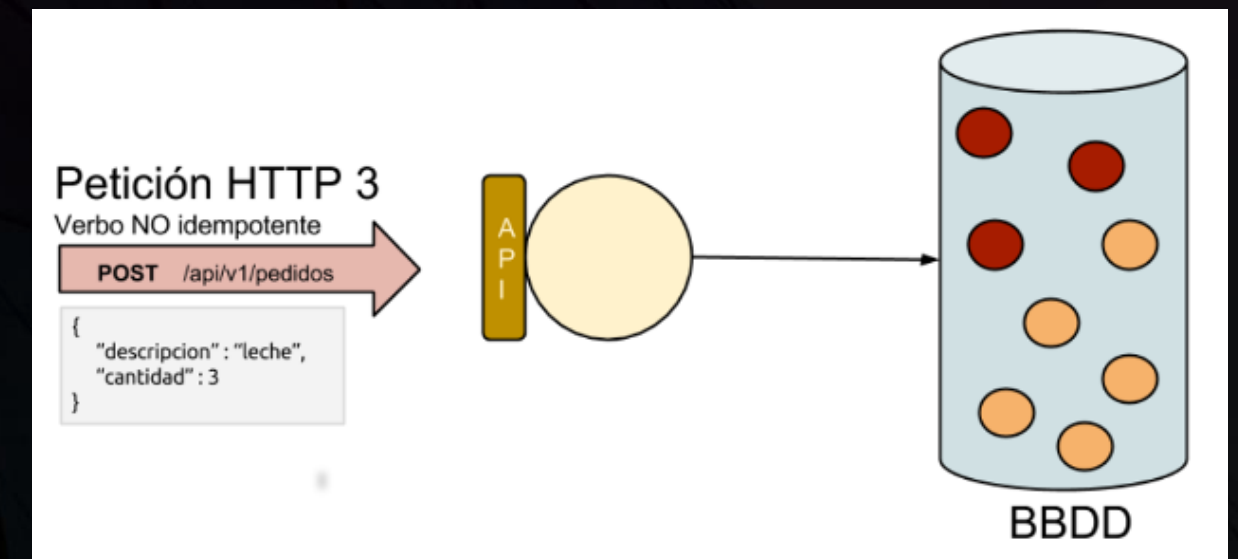
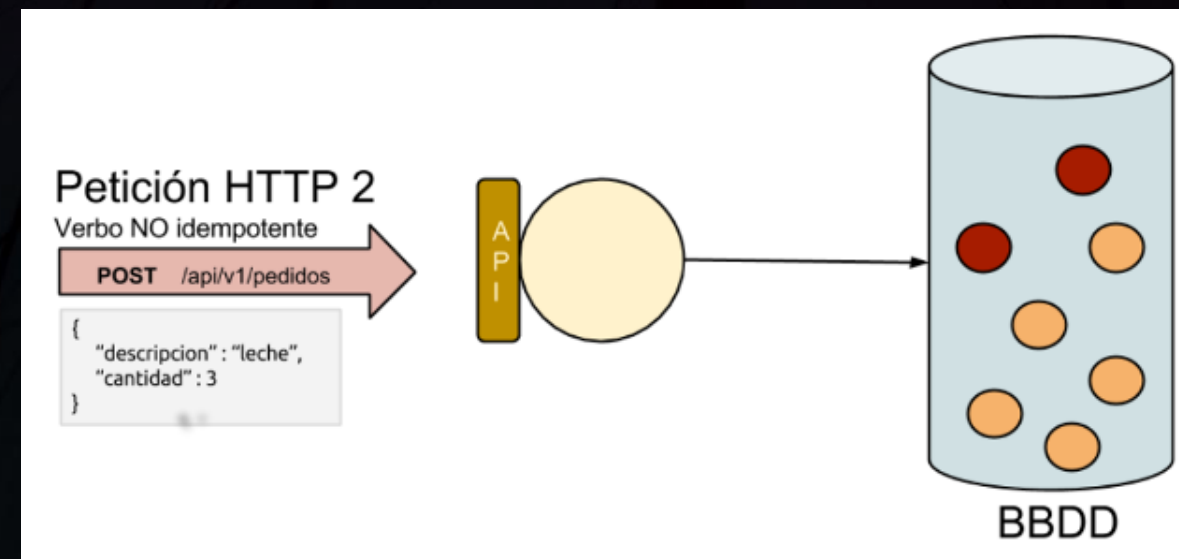
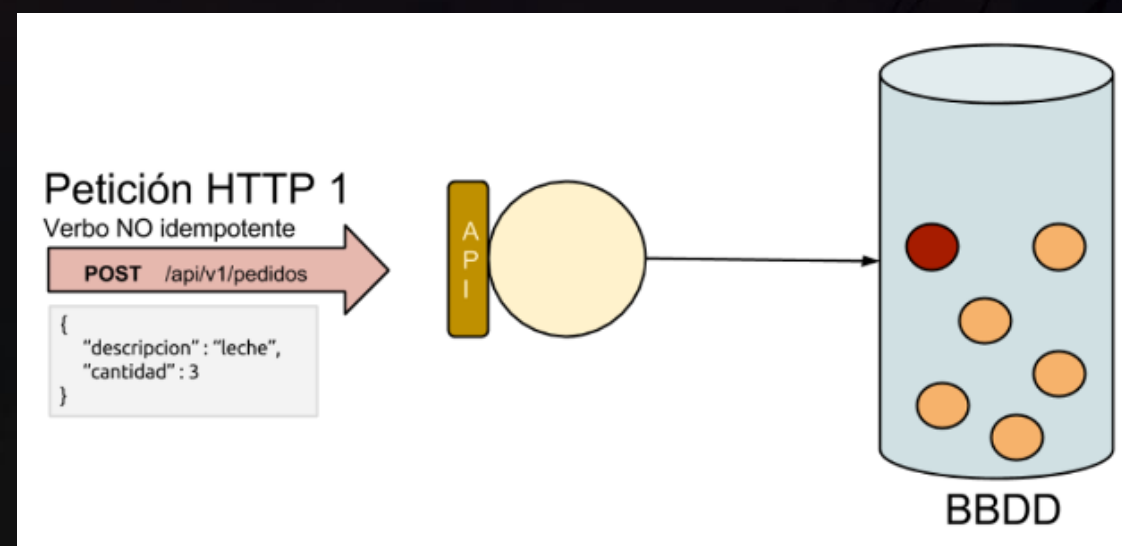
Los metodos pueden ser seguros o inseguros, los seguros son aquellos que nunca modifican un recurso, los inseguros son los que su resultado permite la modificacion de los recursos



Se relacionan al CRUD pero no es el mismo CRUD que se establece en BASE DE DATOS

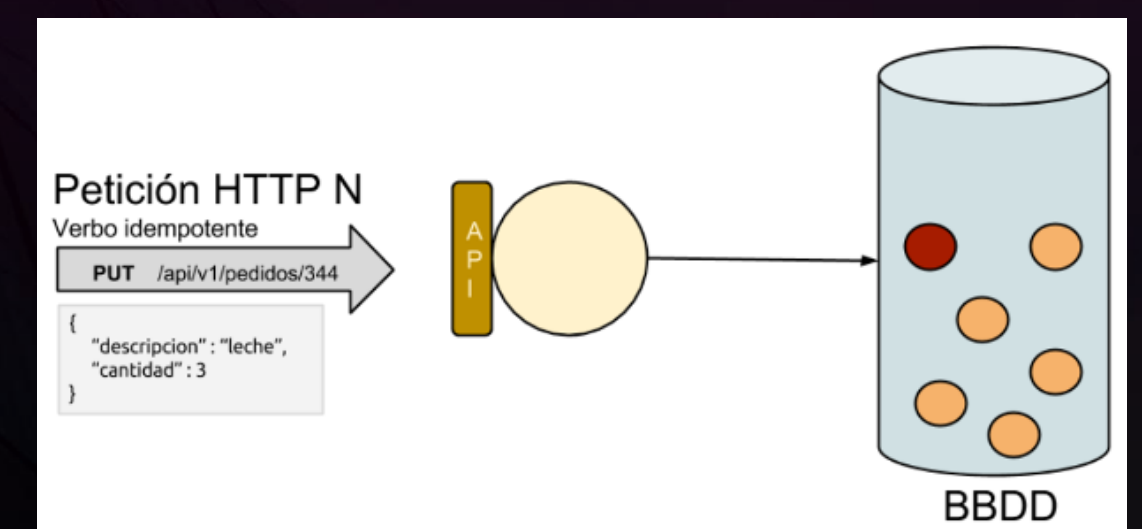
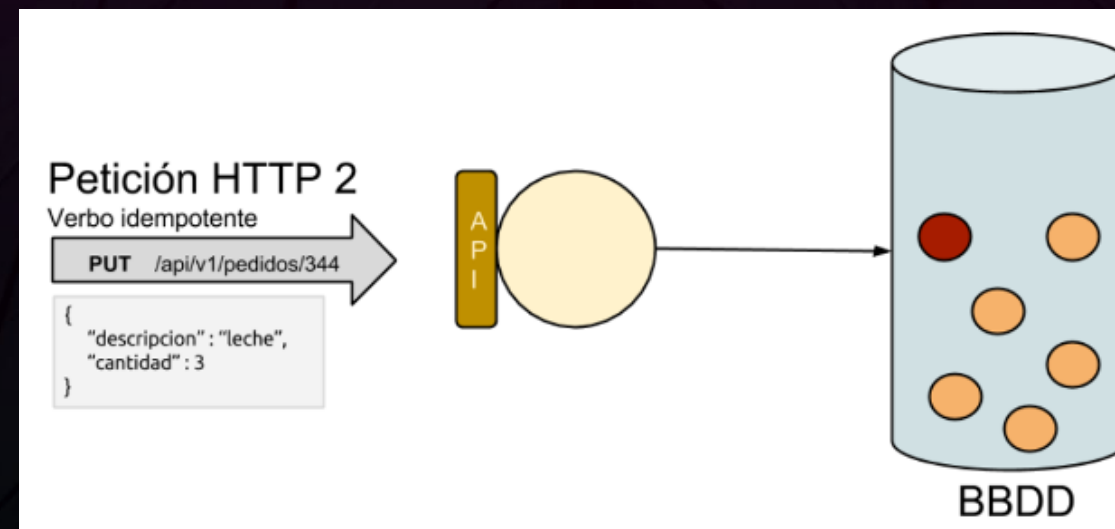
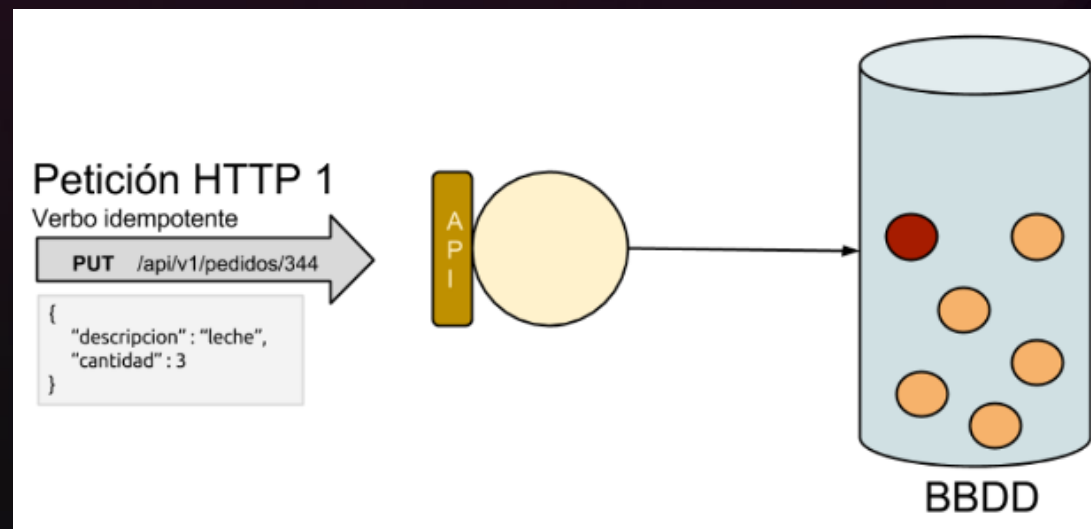
IDEMPOTENCIA

Principio que establece que la ejecución repetida de una petición con los mismo parametros sobre un mismo recurso tendra el mismo efecto en el estado de nuestro recurso en el sistema si se ejecuta 1 o N veces



EL ESTADO DEL SISTEMA CAMBIA SIEMPRE AUNQUE SE APLIQUEN LOS MISMOS PARÁMETROS AL MISMO RECURSO.

IDEMPOTENCIA



ESTE COMPORTAMIENTO
NO SE PRODUCE POR «ARTE
DE MAGIA» POR EL SIMPLE
HECHO DE UTILIZAR UN
VERBO IDEMPOTENTE. ES
NECESARIO IMPLEMENTAR
LA IDEMPOTENCIA EN
NUESTRO SERVICIO.

VERBO	IDEMPOTENTE
GET	SI
POST	NO
PUT	SI
DELETE	SI



RECURSOS - URL

Su función es identificar las cosas en las que se desea operar

Cada URL identifica un recurso, son las mismas URL que se asignan a las páginas web. De hecho, una página es un tipo de recurso

/clients → identifica a todos los clientes

/clients/jim → identifica al cliente llamado JIM

El host aunque es importante para identificar el recurso como único en toda la web, lo antes mencionado son los PATH o la ruta de recursos

los recursos se consideran mejor como sustantivos



PATH REQUEST

GET https://api.ecommerce.com/v1/orders/12345

Metodo HTTP

DOMINIO o base de la API

path que indica el acceso

Es la parte de la URL o ENDPOINT que define el recurso al que se quiere acceder o manipular



BUENAS PRACTICAS

Usar sustantivos en plural para recursos

✗ Malo: `/getUser`, `/createOrder`

✓ Bueno: `/users`, `/orders`

Usar jerarquias de logica en paths: Relacion padre - hijo es decir un recurso dentro de otro

`/users/{userId}/orders`

Mantener constancia en nombres: No mezclar singular/plural y no mezclar snake case con kebab case

`/userProfile` ✗, `/user-profile` ✓



BUENAS PRACTICAS

Evitar profundidad innecesaria

✗ `/users/{userId}/orders/{orderId}/items/{itemId}/details`
✓ `/orders/{orderId}/items/{itemId}`

Usar query params para filtros, busquedas o paginacion no para identificar

✗ `/orders?id=123`
✓ `/orders/123`

`GET /orders?status=shipped&limit=20&page=2`

Mantener los paths predecibles: Si existe `/users/{userId}` deberia existir `/users` y `/users/{userId}/additional-data`



DATOS DE ENTRADA DE APIS

Un request (solicitud) es el mensaje que envía el cliente hacia el servidor para pedir o manipular un recurso.

HEADERS: Metadatos que sirven para configurar las solicitudes

BODY (OPCIONAL): Datos enviados al servidor

```
Content-Type: application/json
Accept: application/json
X-Request-Id: 9f8a7b6c-1234-4567-89ab-9876543210ff

{
  "customerId": "cst-98765",
  "items": [
    {
      "productId": "prd-12345",
      "quantity": 2
    },
    {
      "productId": "prd-67890",
      "quantity": 1
    }
  ],
  "paymentMethod": {
    "type": "card",
    "cardNumber": "4111111111111111",
    "expiryDate": "12/27",
    "cvv": "123"
  },
  "shippingAddress": {
    "street": "Calle 123 #45-67",
    "city": "Bogotá",
    "country": "CO",
    "zipCode": "110111"
  }
}
```



DATOS DE SALIDA DE APIS

Un response (respuesta) es el mensaje que devuelve el servidor al cliente después de procesar el request.

STATUS CODE: Código HTTP del resultado

HTTP/1.1 201 Created

Content-Type: application/json

HEADERS: Información adicional

{

"id": "123",

"name": "Andrés Cantor",

"email": "andres@mail.com",

"createdAt": "2025-09-01T22:00:00Z"

}

BODY (OPCIONAL): Datos recibidos del servidor
(JSON, XML, etc)



JSON VS XML

JSON – JavaScript Object Notation, tiene un formato mas ligero y mas soportable para leer y escribir , es mas eficiente para transmitir datos y es soportado nativamente en varios lenguajes modernos

```
{
  "orderId": "12345",
  "customer": {
    "id": "cst-987",
    "name": "Andrés Cantor"
  },
  "items": [
    { "productId": "prd-111", "quantity": 2 },
    { "productId": "prd-222", "quantity": 1 }
  ],
  "status": "pending"
}
```

XML – Extensible Markup Language, es un formato basado en etiquetas similar a HTML, es mas verbal y pesado al usar tags de apertura y cierre, permite definir atributos en etiquetas, es mas usado en sistemas legados como SOAP

```
<order id="12345">
  <customer>
    <id>cst-987</id>
    <name>Andrés Cantor</name>
  </customer>
  <items>
    <item>
      <productId>prd-111</productId>
      <quantity>2</quantity>
    </item>
    <item>
      <productId>prd-222</productId>
      <quantity>1</quantity>
    </item>
  </items>
  <status>pending</status>
</order>
```



DOSW COMPANY

INVERSION DE DEPENDENCIAS



INVERSION DE DEPENDENCIAS

Establece que las dependencias deben estar en las abstracciones, no en los detalles de la implementacion

Los modulos de alto nivel no deberian depender de modulos de bajo nivel

En lugar de depender directamente de clases concretas, las clases deben depender de interfaces o abstracciones para mayor flexibilidad

CONSEJO DE APLICACION



Definido mediante interfaces lo que ayuda a que la aplicacion nucleo no tendra que conocer la implementacion real para funcionar

Permite tener bajo ACOPLAMIENTO y alta COHESION

```
class Shopping { ... }

class ShoppingBasket {
    fun buy(shopping: Shopping?) {
        val db = SQLiteDatabase()
        db.save(shopping)
        val creditCard = CreditCard()
        creditCard.pay(shopping)
    }
}

class SQLiteDatabase {
    fun save(shopping: Shopping?) {
        // Saves data in SQL database
    }
}

class CreditCard {
    fun pay(shopping: Shopping?) {
        // Performs payment using a credit card
    }
}
```





1

```
interface Persistence {  
    fun save(shopping: Shopping?)  
}  
  
class SQLiteDatabase : Persistence {  
    override fun save(shopping: Shopping?) {  
        // Saves data in SQL database  
    }  
}  
  
interface PaymentMethod {  
    fun pay(shopping: Shopping?)  
}  
  
class CreditCard : PaymentMethod {  
    override fun pay(shopping: Shopping?) {  
        // Performs payment using a credit card  
    }  
}
```

2

```
class ShoppingBasket(  
    private val persistence: Persistence,  
    private val paymentMethod: PaymentMethod  
) {  
    fun buy(shopping: Shopping?) {  
        persistence.save(shopping)  
        paymentMethod.pay(shopping)  
    }  
}
```

3

```
class Server : Persistence {  
    override fun save(shopping: Shopping?) {  
        // Saves data in a server  
    }  
}  
  
class Paypal : PaymentMethod {  
    override fun pay(shopping: Shopping?) {  
        // Performs payment using Paypal account  
    }  
}
```

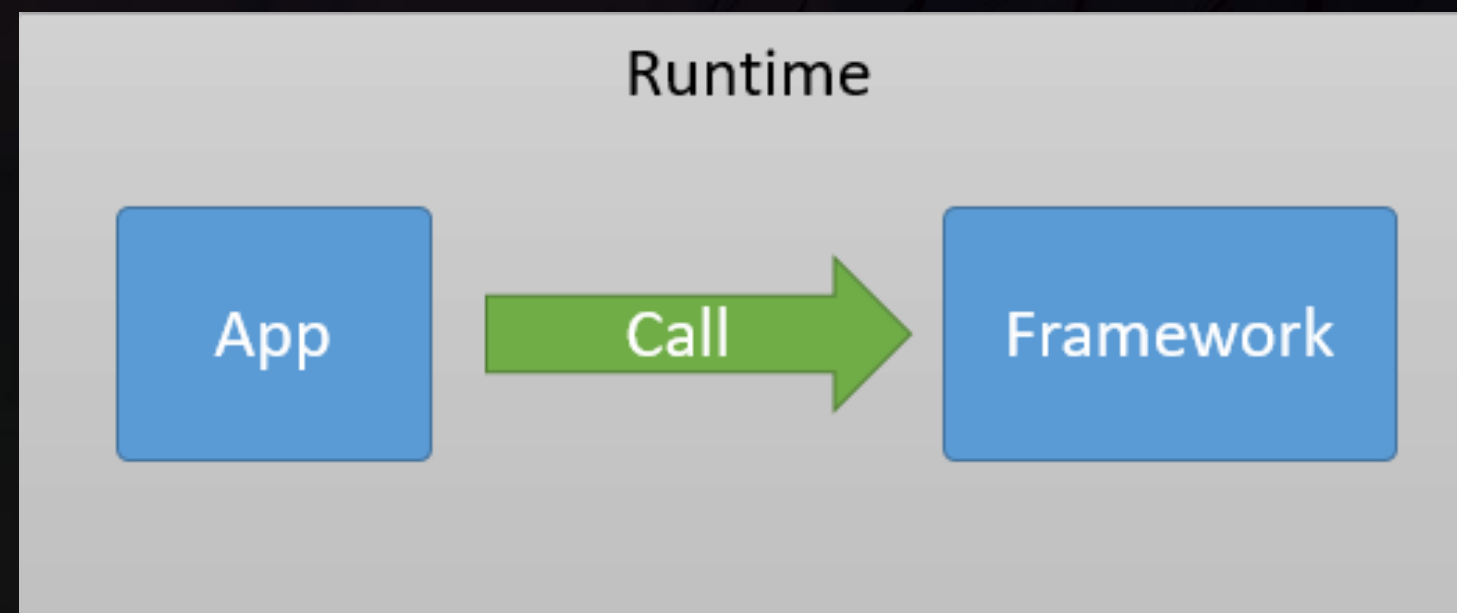


DOSW COMPANY

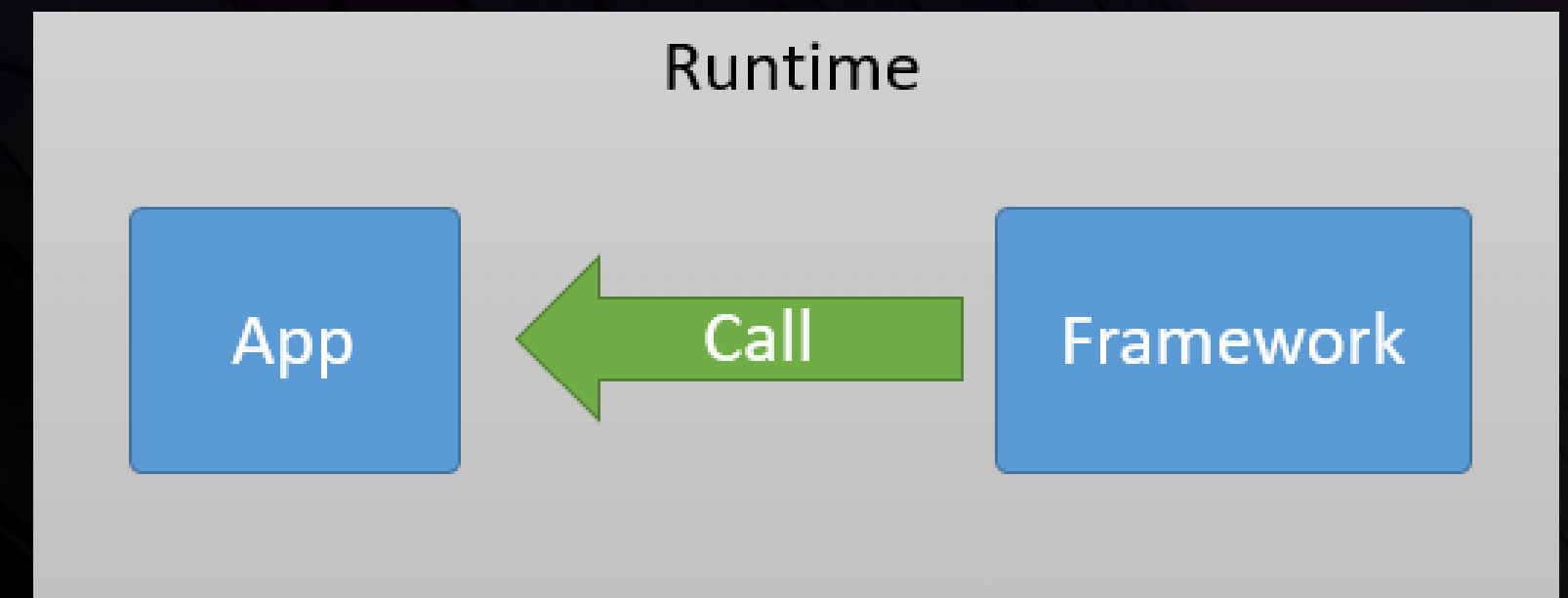
INVERSION DE CONTROL - IOC

INVERSION DE CONTROL - IOC

Es una forma de trabajar que rompe con el formato tradicional, en donde el programador es el encargado de definir la secuencia de operaciones que se deben de realizar para llegar a un resultado.

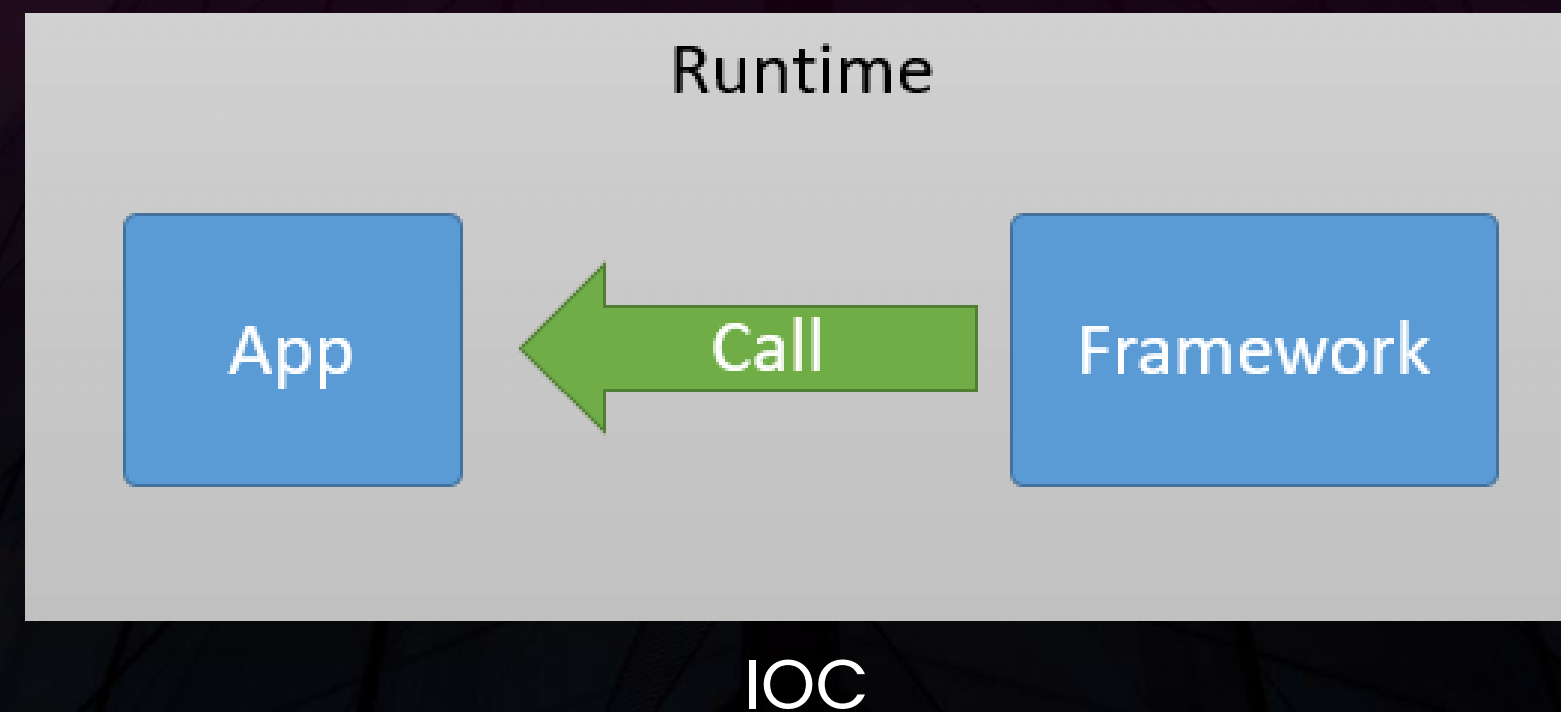


Flujo Normal



IOC

INVERSION DE CONTROL - IOC



Las llamadas se invirtieron, ahora es el Framework es el realiza operaciones sobre nuestra App.

IoC se basa en la introspección (en Java llamado Reflection)



INVERSION DE CONTROL - IOC

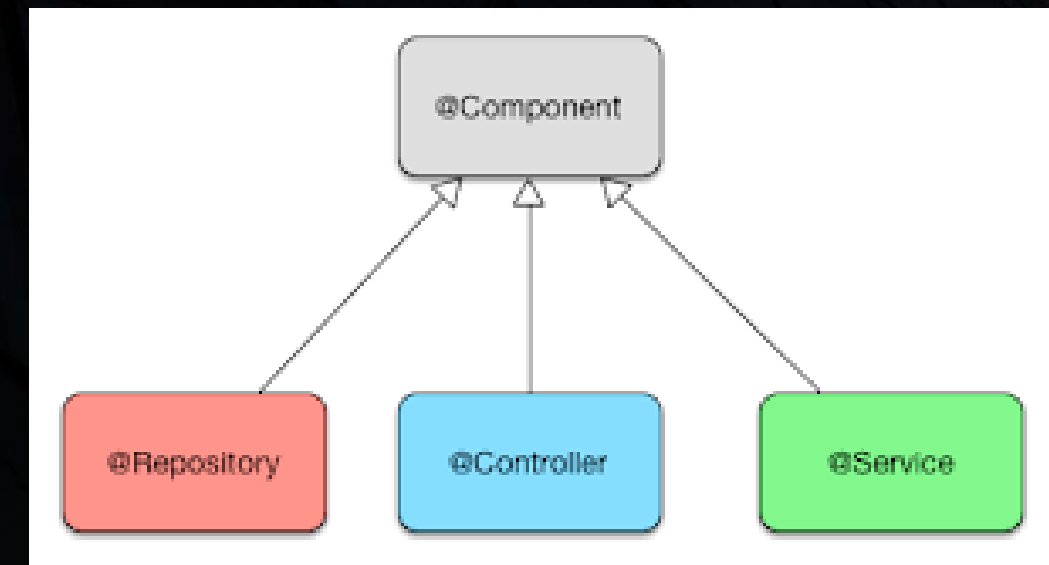
La introspección (REFLECTION) son técnicas para analizar clases en tiempo de ejecución, mediante esta es posible saber las propiedades, métodos, clases que extiende e implementa, ejecutar operaciones, settear u obtener valores, etc. y todo esto sin saber absolutamente nada de la clase.

Es un procedimiento por el cual leemos metadatos de la App que nos permita entender cómo funciona

INVERSION DE CONTROL - IOC

los metadatos pueden estar expresados principalmente de dos formas, la primera es mediante archivos de configuración como XML o con metadatos directamente definidos sobre las clases de nuestro programa, en Java estos metadatos son las @Annotations o anotaciones y es por medio de estos metadatos y con técnicas de introspección que es posible entender el funcionamiento de la App.

```
1 <?xml version = "1.0" encoding = "UTF-8"?>
2
3 <beans xmlns = "http://www.springframework.org/schema/beans"
4     xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
5     xsi:schemaLocation = "http://www.springframework.org/schema/beans
6     http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
7
8     <bean id = "employee" class = "com.flex.reusable.components.model.BeanFactory">
9         <property name = "empName" value = "Muneer Ahmed J"/>
10     </bean>
11
12 </beans>
```





INVERSION DE CONTROL - IOC

IoC evita utilizar el operador new para crear clases clave o que son gestionadas por el Framework. En su lugar el Framework creará estos objetos por nosotros y nos podrá a disposición por medio de inyección de dependencias.

```
package com.oscarblancarte.ioc;

import javax.annotation.Resource;

/**
 * @author Oscar Blancarte
 */
@Service
public class EmployeeService {

    private TransactionManager tran;

    public EmployeeService(){}

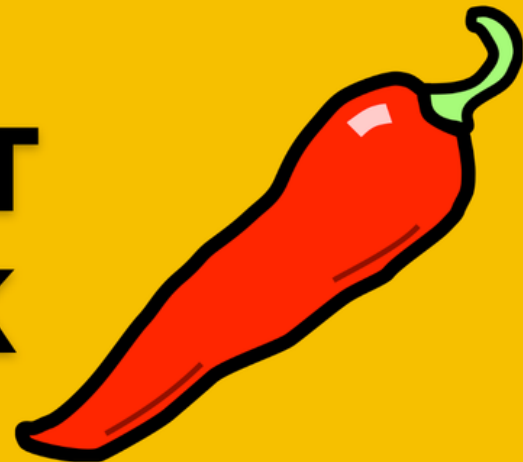
    @Resource
    public void setTransactionManager(TransactionManager tran){
        this.tran = tran;
    }

    @Transaction
    public void saveEmployee(Employee emp){
        tran.persist(emp);
    }
}
```



INVERSION DE CONTROL - IOC

**PROJECT
LOMBOK**





DOSW COMPANY



SPRING BOOT



SPRING BOOT

¿QUE ES?

Es una popular estructura empresarial de código abierto para crear aplicaciones independientes de nivel de producción que se ejecutan en la máquina virtual Java (JVM).

Es una herramienta que hace que el desarrollo de aplicaciones web y microservicios con Spring Framework sea más rápido y fácil a través de tres funcionalidades principales:

CONFIGURACION
AUTOMATICA

1

ENFOQUE
OBSTINADO DE LA
CONFIGURACION

2

CREAR
APLICACIONES
INDEPENDIENTES

3



SPRING BOOT

Ofrece una función de inyección de dependencias que permite que los objetos definan sus propias dependencias que el contenedor de Spring luego inyecta en ellos.

Esto permite a los desarrolladores crear aplicaciones modulares que constan de componentes poco acoplados



SPRING BOOT ANOTACIONES



ANOTACIONES INICIALES EN SPRING BOOT

`@Component` → Marca una clase genérica como componente de Spring.

`@Service` → Especialización de `@Component`, semánticamente usada en la capa de servicio.

`@Repository` → Especialización de `@Component`, usada en la capa de acceso a datos (DAO). Incluye manejo automático de excepciones de persistencia.

`@Controller` → Usada en controladores MVC (manejo de vistas).

`@RestController` → Combina `@Controller` + `@ResponseBody`, ideal para APIs REST.

`@Autowired` → Inyección automática de dependencias (por tipo).

`@Qualifier("beanName")` → Especifica qué bean usar cuando hay múltiples candidatos.

`@Value("${property}")` → Inyecta valores desde `application.properties` o `application.yml`.

`@Inject` (JSR-330) → Similar a `@Autowired`.

`@Resource` (JSR-250) → Inyección por nombre (más común en proyectos antiguos).

`@Configuration` → Clase que define beans de configuración.

`@Bean` → Método dentro de una clase `@Configuration` que expone un bean al contenedor.

`@Scope("singleton" | "prototype" | ...)` → Define el alcance del bean.

`@Lazy` → Crea el bean solo cuando se necesite (lazy loading).

`@RequestMapping("/ruta")` → Define rutas de endpoints HTTP.

`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` → Atajos de `@RequestMapping` para métodos HTTP específicos.

`@PathVariable` → Obtiene parámetros de la URL.

`@RequestParam` → Obtiene parámetros de consulta (`?param=valor`).

`@RequestBody` → Mapea el cuerpo de la petición a un objeto Java.

`@ResponseBody` → Retorna datos en vez de vistas (usado en APIs).

`@CrossOrigin` → Habilita CORS en un endpoint.

`@SpringBootTest` → Levanta el contexto completo de la app para pruebas.

`@WebMvcTest` → Carga solo la capa web para pruebas de controladores.

`@DataJpaTest` → Configura un entorno de prueba para repositorios JPA.

`@MockBean` → Crea mocks gestionados por Spring.



ACTIVIDAD

1. Creen su Repositorio – Bitacora_Corte2_NombreApellido
 - a. Debe tener pom.xml y Maven
 - i. Agreguen las dependencias de pruebas y las de springboot y lombok
 - b. README con el nombre y descripcion de su restaurante
 - c. Agreguen en el readme los diagramas que hicimos hoy
 - d. Creen la estructura de carpetas – Con clases vacias

EXPOSICIONES PROYECTO

Deben tener todo preparado - Segun cronograma SEMANA

7
Feedback y retroalimentación - 15 minutos (5 minutos)





TAREA

1. Recuerden terminar el taller de Mocks generados con I.A
Tienes hasta hoy a media noche para entregarlo
2. Esta disponible el taller TDD en SEMANA 7 – Por favor
desarrollarlo individualmente.
3. Investiguen – La anotaciones de springboot y su uso
4. Terminen de subir la estructura inicial de su Proyecto del
restaurante



NOS VEMOS EN LA



PROXIMA SESION DE LABORATORIO - REVISION INICIAL DEL PROYECTO